

UNIT-4: Python Interaction with text and CSV

4.1 File handling (text and CSV files) using CSV module:

4.1.1 CSV module

- For working csv (Comma Separated Value) files using python, CSV module is available.
- There is a need to import CSV module into python for working with CSV files.
- Syntax:

```
import csv
```
- This module allows multiple methods which can help to read data from CSV file and to write data into CSV file through python.

4.1.2 File access modes: Read, write, append

- File access modes allows types of operations which are possible with opened file.
- It is used to specify “How the file will be used when it is opened?”.
- Following are the modes of accessing files:
 - Read ('r'):
 - It opens file for reading only.
 - User can't write data when the file is opened as READ access.
 - It set the pointer at the beginning of the opened file.
 - It generates error if the file is not available.
 - Write ('w'):
 - It opens file for writing only.
 - If the file is not available then it creates new file.
 - If the file is available then it sets the pointer at the beginning of the opened file and it over writes data. (Loss of data)
 - Append ('a'):
 - It opens file for writing only.
 - If the csv file is not available then it creates new file.
 - If the csv file is available then set the pointer at the end of the file. (No loss of data)
 - The data is inserted at the end means after existing data.
 - Read and write ('r+'):
 - It opens file for reading and writing.
 - It set the pointer at the beginning of the file.
 - It generates error if the file does not exist.
 - Append and read ('a+'):
 - It opens file for reading and writing.
 - If the file is not available then it is created.
 - It sets the pointer at the end of the file.
 - The data is inserted at the end means after existing data.

4.2 Important Classes and Functions of CSV modules:

4.2.1 Open() function:

- Python allows open() function to open file.
- Syntax:

```
File_object=open(r "file_name", "Access mode")
```
- File name can be of local directory or you can set the path of the file.
- If the file is in local directory, then 'r' is not required.
- If the file is at another location, then you can write 'r'. (Sometime face error because of '\'.)
- 'r' makes the string raw. It specifies that the specified filename is string only where no special character available.
- Examples:

Example 1:

If test.csv file is in local directory and we want to open file in write mode then....

```
f1=open("test.csv","w")
```

Example 2:

If temp.csv file is in SQLITE directory and want to open file in append mode then...

```
f2=open(r "c:\sqlite\temp.csv","a")
```

4.2.2 Close() function:

- To close opened file, close() function is available.
- Syntax:

```
File_object.close()
```
- Example:

```
f1.close()
```

4.2.3 WITH statement:

- WITH statement is used for code cleaner and for make the code more readable.
- If WITH statement is used then it automatically releases all resources. So there is no need to close the file externally. It automatically close the file.
- Syntax:

```
with open_function as file_object:
    statements
```

- Example:

```
with open("test.csv","r") as f1:
```

```
    a=f1.reader(f1)
```

4.2.4 reader() function:

- CSV module allows reader() function (or method) through which the data can be read from CSV file.
- csv.reader() is an iterative (repetitive) function.
- Syntax:

```
Reader_object=csv.reader(file_object, optional_parameters)
```

- It returns READER object which helps to display data of each row through loop.
- CSV file:

```
Rollno,Name,Gender,Percentage
101,Nency,Female,67
102,Het,Male,48
103,Maya,Female,78
104,Jeet,Male,69
105,Jitesh,Male,80
106,Nayan,Male,44
107,Geeta,Female,65
108,Manthan,Male,90
109,Jivan,Male,45
110,Preet,Female,87
```

- Example:

```
import csv
with open('stud.csv','r') as f1:
    r1=csv.reader(f1)
print('-'*60)
for row in r1:
    #print(row)      To print single row
    print('%10s'%row[0],'%15s'%row[1],'%15s'%row[2],'%10s'%row[3])
print('-'*60)
```

Output:

Rollno	Name	Gender	Marks
134	Maya	Female	45
135	Manthan	Male	34
136	Mohit	Male	33

- Csv.reader() is an iterative (repetitive) function so “next(reader_object)” method returns current row and set the iterator to next row.

- Example:

(Display detail of students having more than 50 percentage.)

```
import csv
with open("stud.csv","r") as f1:
    r1=csv.reader(f1,delimiter=',')
    col=next(r1)
print('-'*60)
print(col)
print('-'*60)

for row in r1:
    if int(row[3])>50:
```

```
print(row)
```

```
-----
['Rollno', 'Name', 'Gender', 'Percentage']
-----
['101', 'Nency', 'Female', '67']
['103', 'Maya', 'Female', '78']
['107', 'Geeta', 'Female', '65']
['110', 'Preet', 'Female', '87']
```

- Line_num is used to return the current line number available in CSV file.

Example: r1.line_num

(Display total numbers of lines in csv file.)

```
import csv
with open("stud.csv","r") as f1:
    r1=csv.reader(f1,delimiter=',')
    print(r1.line_num)
    col=next(r1)
print('-'*60)
    print(col)
print('-'*60)
    print(r1.line_num)
    for row in r1:
        if row[2]=="Female":
            print(row)
    print(r1.line_num)
```

Output:

```
0
-----
['Rollno', 'Name', 'Gender', 'Percentage']
-----
1
['101', 'Nency', 'Female', '67']
['103', 'Maya', 'Female', '78']
['107', 'Geeta', 'Female', '65']
['110', 'Preet', 'Female', '87']
11
```

4.2.5 writer() function:

- CSV module allows writer() function (or method) through which the data can be written in CSV file.
- Syntax:
writer_object=csv.writer(file_object, optional_parameters)
- WRITEROW() and WRITEROWS() functions are used to write data into file.
- Example:

```
import csv
cols=['rno', 'name', 'gender', 'per']
rows=[[1,'Atish','Male',78],[2,'Ankur','Male',56],
      [3,'Ankita','Female',87],[4,'Akash','Male',90],
      [5,'Aniket','Male',50]]
with open("student.csv","w") as f1:
    w1=csv.writer(f1)    #by default '\r\n'
```

```
w1.writerow(cols)
w1.writerows(rows)
```

Output:

rno	name	gender	per
1	Atish	Male	78
2	Ankur	Male	56
3	Ankita	Female	87
4	Akash	Male	90
5	Aniket	Male	50

- By default line-terminator is '\r\n'. Because of \r it gives one separate line after each line. So set lineterminator to '\n' to solve above mentioned line spacing.
- Example:

```
import csv
cols=['rno', 'name', 'gender', 'per']
rows=[[1,'Atish','Male',78],[2,'Ankur','Male',56],
      [3,'Ankita','Female',87],[4,'Akash','Male',90],
      [5,'Aniket','Male',50]]
with open("student.csv","w") as f1:
    w1=csv.writer(f1,lineterminator='\n')
    w1.writerow(cols)
    w1.writerows(rows)
```

Output:

rno	name	gender	per
1	Atish	Male	78
2	Ankur	Male	56
3	Ankita	Female	87
4	Akash	Male	90
5	Aniket	Male	50

4.2.6 writerow() function:

- writerow() takes one dimensional data to write in CSV file.
- It writes list of values in single line into given csv file.
- Syntax: writer_object.writerow(values)
- Example: w1.writerow(cols)

4.2.7 Writerows() function:

- Writerows() takes two dimensional data to write in CSV file.
- It writes list of values in multiple lines at once into given csv file.
- Syntax: writer_object.writerows(values)
 - Example: w1.writerows(rows)

4.2.8 DictReader() function:

- CSV module allows DictReader() function through which the data can be read from CSV file in form of python dictionary.
- It creates CSV object which works like python OrderedDict.
- It considers first row's each elements as dictionary key and all other rows' each elements as dictionary values. Each value can be accessed using dictionary key.
- Syntax:
Reader_object=csv.DictReader(file_obj, optional_parameters)
- Example:

```
import csv
with open('stud.csv','r') as f1:
    r1=csv.DictReader(f1)
    for i in r1:
        print(i)
```

4.2.9 DictWriter()

- CSV module allows DictWriter() function through which the data can be written into CSV file using python dictionary.
- Syntax:
Writer_object=csv.DictWriter(file_obj, optional_parameters)
- Example:

```
import csv
cols=['rno','name']
rows=[{'rno':1,'name':'Het'},{'rno':2,'name':'Hetvi'}]
with open('student.csv','w') as f1:
    w1=csv.DictWriter(f1,fieldnames=cols,lineterminator='\n')
    w1.writeheader()
    w1.writerows(rows)

or
import csv
cols=['rno','name']
with open('student.csv','w') as f1:
    w1=csv.DictWriter(f1,fieldnames=cols, lineterminator='\n')
    w1.writeheader()
    w1.writerow({'rno':1,'name':'Het'})
    w1.writerow({'rno':2,'name':'Hetvi'})
```

4.3 NumPy in python

- It is used to create an array in python.
- NumPy provides “array” function through which the array can be created.
- Example of creating array using numpy:

```
import numpy as np
#one dimensional array
a=np.array([1,2,3,4,5])
print(a)
print("Third value is ",a[2])

#two dimensional array
b=np.array([[1,2,3],[4,5,6]])
print(b)
print("First row and 2nd value is ",b[1,2])

#three dimensional array
c=np.array([[[1,2,3],[4,5,6]], [[7,8,9],[10,11,12]]])
print(c)
print("1,2,2nd value: ",c[0,1,1])
```

Output:

```
[1 2 3 4 5]
Third value is 3
[[1 2 3]
 [4 5 6]]
First row and 2nd value is 6
[[[ 1  2  3]
   [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
1,2,2nd value: 5
```

- To check the dimension:

```
print(a.ndim)
print(b.ndim)
print(c.ndim)
```

Output:

```
1
2
3
```

- Slicing of an array:
 - Specify the first and (last+1) number to display specific values of an array.
 - Syntax: array_name[start_no:(end_no+1):skip_no]
 - Example:

```
print(a)
print(a[1:3])
print(a[1:])
print(a[:3])
print(a[0:5:2])
print(a[5:0:-1])
```

Output:

```
[1 2 3 4 5]
[2 3]
[2 3 4 5]
[1 2 3]
[1 3 5]
[5 4 3 2 1]
```

- Example: Create following table using numpy:

Rno	Name	Marks
1	Ashish	67
2	Mahesh	56
3	Jatin	76

```
import numpy as np
cols=['Rno','Name','Marks']
a=np.array([[1,'Ashish',67],[2,'Mahesh',56],[3,'Jatin',76]])
print(cols)
print(a)
```

4.4 Pandas in python

- Pandas is a library which allows to perform data manipulation and analysis in python.
- It offers data manipulation and analysis related operations for numerical tables.
- It needs NumPy package for process.

4.5 DataFrame in pandas

- It is main object in Pandas.
- It is used to represent data with rows and columns (tabular form)
- It is 2D array with different labels and different data types.
- It displays the data in tabular form.
- It includes three components:
 - o Data: It includes all values.
 - o Row index: It includes all rows
 - o Column label: It includes all columns.

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56

- Here 0 to 4 are row_index, (rno,name,marks) are column_labels and all others are the values.
- Syntax for creating dataframe:
Panda_object.DataFrame(data=values, index=ind_values, columns=col_values)
- Here
 - Data represents actual values
 - Index represents row_index
 - Columns represents column_labels.

4.5.1 Creating dataframe from numpy array:

- NumPy allows us to create array. Using numpy, we can create tuples (rows).
- Example:

```
import pandas as pd
import numpy as np
a=np.array([[1,'Het',45],[2,'Raj',67]])
row=[1,2]
col=['rno','name','marks']
df=pd.DataFrame(data=a, index=row, columns=col)
print(df)
```

Output:

```
rno name marks
1 1 Het 45
2 2 Raj 67
```

4.5.2 Creating dataframe using python dictionary: (Not in Syllabus)

- Dataframe can be created using python dictionary.
- Example:

```
import pandas as pd
a={'rno':[1,2],
  'name':['Het','Raj'],
  'marks':[45,67]
}
df=pd.DataFrame(a)
print(df)
```

4.5.3 Creating dataframe using CSV file (Reading from CSV file using Pandas)

- Pandas provides `read_csv()` to read CSV file and to do operation on it.
- Dataframe is created when the data is read using `read_csv()`.
- Syntax of `read_csv()`:
Dataframe_obj=pandas_obj.read_csv("filename",optional_parameters)
- Example:

```
import pandas as pd
df = pd.read_csv('test_csv.csv')
print(df)
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56

```
import pandas as pd
df = pd.read_csv('test_csv.csv',index_col='name')
print(df)
```

Output:

	rno	marks
name		
Hiral	1	56
Twinkle	2	76
Naina	3	87
Kejal	4	67
Mansi	5	56

- Parameters:
 - o `Index_col='col_name'`: It is used to set the index to specific field of CSV file.
 - o `header=None`: It is required to set when the header is not available in the CSV file.
 - o `header=no`: It is required to set when you want to tell that the header is available in specific row. Ex: `header=1`
 - o `skiprows=no`: It is required to set when you want to skip numbers of rows from the top. (If it is blank) Ex: `skiprows=1`
 - o `names=[field names]`: It is set when the header is not available in CSV file and you want to set the field names into Dataframe.
 - o `nrows=no`: It is set when you want to read few rows from CSV file in place of all rows.

4.5.4 Writing into CSV file using Pandas

- Pandas provides `to_csv()` to write into CSV file.
- It writes data which is available in dataframe.

- Syntax of to_csv():
Dataframe_obj.to_csv("filename",optional_parameters)

- Example:

```
import pandas as pd
a={'rno':[1,2],
  'name':['Het','Raj'],
  'marks':[45,67]
}
df=pd.DataFrame(a)
df.to_csv('temp.csv')
```

Output:

	rno	name	marks
0	1	Het	45
1	2	Raj	67

- When we write dataframe data into CSV file then it also writes the row_index into file.
- Optional_Parameters:
 - o index=False: It doesn't write index column into CSV file.
 - o columns=[field names]: It writes specific columns only into CSV file.

4.5.5 Creating dataframe using Excel file (Read data from Excel file using pandas)

- Pandas provides read_excel() to read Excel file and to do operation on it.
- Dataframe is created when the data is read using read_excel().
- Openpyxl is additionally required for importing data from Excel file.
- Syntax:
Dataframe_obj=pandas_obj.read_excel("filename","Sheet_name",opt_parameters)
- Example:

```
import pandas as pd
df = pd.read_excel ('test.xlsx','Sheet1')
print(df)
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56
5	6	Rajvi	89
6	7	Mona	78

```
import pandas as pd
df = pd.read_excel ('test.xlsx','Sheet1',index_col='marks')
print(df)
```

Output:

marks	rno	name
56	1	Hiral
76	2	Twinkle
87	3	Naina
67	4	Kejal
56	5	Mansi
89	6	Rajvi
78	7	Mona

4.5.6 Writing into Excel file using Pandas

- Pandas provides to_excel() to write into excel file.
- It writes data which is available in dataframe.
- Syntax:
- Dataframe_obj.to_excel("filename",optional_parameters)
- Optional_Parameters:
 - o sheet_name='name': It sets the worksheet name as given here. If it is not specified then it set the worksheet name as 'Sheet1'
 - o index=False: It doesn't write index column into CSV file.
 - o columns=[field names]: It writes specific columns only into CSV file.
 - o startrow=no: To start writing data from specific row.
 - o startcol=no: To start writing data from specific column.
- Example:

```
import pandas as pd
df = pd.read_excel('test.xlsx','Sheet1')
print(df)
df.to_excel('temp.xlsx',sheet_name='Student',index=False,columns=['rno','name'])
```

Output:

rno	name
1	Hiral
2	Twinkle
3	Naina
4	Kejal
5	Mansi
6	Rajvi
7	Mona

4.5.7 Operations on dataframe:

- Working with following dataframe:
- Example:

```
import pandas as pd
df = pd.read_excel('test.xlsx','Sheet1')
print(df)
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56
5	6	Rajvi	89
6	7	Mona	78

4.5.7.1 *shape*

- It represents the dimension of dataframe.
- Example:

```
print(df.shape)
```

Output:

```
(7,3)
```

- To store the dimension's values into specific variables
- Example:

```
r,c=df.shape
```

```
print("No. of rows: ",r)
```

```
print("No. of columns: ",c)
```

Output:

```
No. of rows: 7
```

```
No. of columns: 3
```

4.5.7.2 *columns*

- Df.columns displays the list of fields available in given dataframe.
- Example:

```
print(df.columns)
```

Output:

```
Index(['rno', 'name', 'marks'], dtype='object')
```

4.5.7.3 *Extracting specific rows from dataframe*

- To display rows as per condition from dataframe, follow following syntax.
- Syntax:

```
Df_obj[condition]
```

- Example:

```
print(df[df.marks>80])
```

Output:

	rno	name	marks
2	3	Naina	87
5	6	Rajvi	89

```
print(df[(df.marks>70) & (df.rno>2)])
```

Output:

```

    rno    name  marks
2     3   Naina    87
5     6   Rajvi    89
6     7    Mona    78

print(df['marks'].min())
Output:
56

print(df.marks.sum())
Output:
509

print(df[df.marks==df.marks.max()])
Output:
    rno    name  marks
5     6   Rajvi    89

df1=df[df.name=='Naina']
print(df1)
Output:
    rno    name  marks
2     3   Naina    87

```

4.5.7.4 Extract specific attributes (Columns) from dataframe

- To display specific fields information, write following syntax:
Df_obj[['field_name1','field_name2',...]]
- Example:

```

print(df[['rno','name']])
Output:
    rno    name
0     1   Hiral
1     2  Twinkle
2     3   Naina
3     4   Kejal
4     5   Mansi
5     6   Rajvi
6     7    Mona

print(df['name'][df.marks>70])
Output:
1     Twinkle
2     Naina
5     Rajvi
6     Mona
Name: name, dtype: object

```

4.5.8 Dataframe functions:

4.5.8.1 *Head*

- It is used to display few rows of dataframe from starting position.
- Syntax:
df_obj.head(n)
- head() function displays first five rows of dataframe.
- head(n) function displays first n rows of dataframe.
- Example:

```
print(df.head())
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56

```
print(df.head(2))
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76

4.5.8.2 *tail*

- It is used to display few rows of dataframe from end.
- Syntax:
df_obj.tail(n)
- tail() function displays last five rows of dataframe.
- tail(n) function displays last n rows of dataframe.
- Example:

```
print(df.tail())
```

Output:

	rno	name	marks
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56
5	6	Rajvi	89
6	7	Mona	78

```
print(df.tail(2))
```

Output:

	rno	name	marks
5	6	Rajvi	89
6	7	Mona	78

4.5.8.3 To display specific rows from the middle of dataframe:

- Syntax: `df_obj[start:(end+1)]`
- Example:

```
print(df[2:5])
```

Output:

	rno	name	marks
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56

```
print(df[4:])
```

Output:

	rno	name	marks
4	5	Mansi	56
5	6	Rajvi	89
6	7	Mona	78

```
print(df[:4])
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67

4.5.8.4 `describe()`

- It is used to display the statistics for given dataframe.
- It calculates statistical functions for numerical data fields.
- It calculates means, standard deviation, minimum, maximum and percentile values for specified numerical data fields of dataframe.
- Syntax: `df_obj.describe()`
- Example:

```
print(df.marks.describe())
```

Output:

```
count    7.000000
mean     72.714286
std      13.536547
min      56.000000
25%      61.500000
50%      76.000000
75%      82.500000
max      89.000000
Name: marks, dtype: float64
```



```
p=[.10,.20,.30,.40,.50,.70,.90]
print(df.marks.describe(percentiles=p))
```

Output:

```
count      7.000000
mean       72.714286
std        13.536547
min        56.000000
10%        56.000000
20%        58.200000
30%        64.800000
40%        70.600000
50%        76.000000
70%        79.800000
90%        87.800000
max        89.000000
Name: marks, dtype: float64
```

4.5.8.5 Values

- It is used to represent the dataframe in form of NumPy array.
- Syntax: `df_obj.values`
- Example:

```
print(df.values)
```

Output:

```
[[1 'Hiral' 56]
 [2 'Twinkle' 76]
 [3 'Naina' 87]
 [4 'Kejal' 67]
 [5 'Mansi' 56]
 [6 'Rajvi' 89]
 [7 'Mona' 78]]
```

4.5.8.6 to_numpy()

- It is used to return the dataframe in form of NumPy array.
- Syntax: `df_obj.to_numpy()`
- Example:

```
print(df.to_numpy())
```

Output:

```
[[1 'Hiral' 56]
 [2 'Twinkle' 76]
 [3 'Naina' 87]
 [4 'Kejal' 67]
 [5 'Mansi' 56]
 [6 'Rajvi' 89]
 [7 'Mona' 78]]
```

4.5.8.7 Loc

- To extract numbers of rows from dataframe, LOC method is used.
- Loc method takes only Index_labels and returns respective data frame.
- It searches rows as per index value.
- If the search index value has multiple rows then it returns multiple rows.
- Syntax:

```
Df_obj.loc[index_labels]
```

- Example:

```
# To display 5th row
import pandas as pd
df = pd.read_excel('test.xlsx','Sheet1')
print(df,'\n')
print(df.loc[5])
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56
5	6	Rajvi	89
6	7	Mona	78

```
rno      6
name     Rajvi
marks    89
Name: 5, dtype: object
```

To display information of rollno 5.

```
import pandas as pd
df = pd.read_excel('test.xlsx','Sheet1',index_col='rno')
print(df,'\n')
print(df.loc[5])
```

Output:

	name	marks
rno		
1	Hiral	56
2	Twinkle	76
3	Naina	87
4	Kejal	67
5	Mansi	56
6	Rajvi	89
7	Mona	78

```
name     Mansi
marks    56
Name: 5, dtype: object
```

#To display information of students having 56 marks.

```
import pandas as pd
df = pd.read_excel('test.xlsx','Sheet1',index_col='marks')
print(df,'\n')
print(df.loc[56])
```

Output:

	rno	name
marks		
56	1	Hiral
76	2	Twinkle
87	3	Naina
67	4	Kejal
56	5	Mansi
89	6	Rajvi
78	7	Mona

	rno	name
marks		
56	1	Hiral
56	5	Mansi

#To display students' detail having either 67 or 89 marks.

```
import pandas as pd
```

```
df = pd.read_excel('test.xlsx','Sheet1',index_col='marks')
```

```
print(df.loc[[67,89]])
```

Output:

	rno	name
marks		
67	4	Kejal
89	6	Rajvi

#To display students' records having name between Twinkle and Kejal.

```
import pandas as pd
```

```
df = pd.read_excel('test.xlsx','Sheet1',index_col='name')
```

```
print(df,'\n')
```

```
print(df.loc['Twinkle':'Kejal'])
```

Output:

	rno	marks
name		
Hiral	1	56
Twinkle	2	76
Naina	3	87
Kejal	4	67
Mansi	5	56
Rajvi	6	89
Mona	7	78

	rno	marks
name		
Twinkle	2	76
Naina	3	87
Kejal	4	67

4.5.8.8 Iloc

- Iloc means Integer Loc.
- To extract numbers of rows from dataframe, ILOC method is used.
- Iloc method takes only Index position and returns respective data frame.

- It searches rows as per index position.
- When the index label is not known to user and want to search the data then iloc method is used in place of loc method.
- When the data is searched with its default index (i.e. 0,1,2,...) then LOC and ILOC methods return same output.

- Syntax:

Df_obj.iloc[index_position]

- Example:

```
import pandas as pd
df = pd.read_excel('test.xlsx','Sheet1')
print(df,'\n')
```

Output:

	rno	name	marks
0	1	Hiral	56
1	2	Twinkle	76
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56
5	6	Rajvi	89
6	7	Mona	78

To display 5th row

```
print(df.iloc[5])
```

Output:

rno	6
name	Rajvi
marks	89

Name: 5, dtype: object

To display 1,2,3,4 rows

```
print(df.iloc[[2,3,4,5]])
```

Output:

	rno	name	marks
2	3	Naina	87
3	4	Kejal	67
4	5	Mansi	56
5	6	Rajvi	89

To display 3 to 6 rows

```
print(df.iloc[3:6])
```

Output:

	rno	name	marks
3	4	Kejal	67
4	5	Mansi	56
5	6	Rajvi	89

```
# If the index is set to Marks then what is the output with ILOC
import pandas as pd
df = pd.read_excel('test.xlsx','Sheet1',index_col='marks')
print(df,'\n')
print(df.iloc[5])
```

Output:

	rno	name
marks		
56	1	Hiral
76	2	Twinkle
87	3	Naina
67	4	Kejal
56	5	Mansi
89	6	Rajvi
78	7	Mona


```
rno          6
name      Rajvi
Name: 89, dtype: object
```

4.5.9 Central tendency measures:

- Measure of central tendency is the essential part of statistics.
- It is used to summarize the dataset.
- It provides accurate description of entire data.
- Following are commonly used measures:

4.5.9.1 Mean

- Mean() is used to calculate the arithmetic mean or average of given dataset and for requested AXIS.
- Syntax:
Variable_name = DataFrame.mean(axis=[0/1])
- AXIS is the parameter which is used to specify row or column. 0 means calculating mean column wise and 1 means calculating mean row wise. Default axis is 0.
- Example:

```
# Calculating mean row and columnwise.
import pandas as pd
a=[['Mahesh',67,65],['Ramesh',77,45],['Yogesh',67,45],['Pritesh',45,34],
['Jitesh',56,67],['Pragnesh',67,78],['Mantresh',45,56],['Khanesh',87,89],
['Antresh',44,77],['Mitesh',67,89]]
df=pd.DataFrame(data=a,columns=['Name','Sub1','Sub2'])
print(df)
```

Output:

	Name	Sub1	Sub2
0	Mahesh	67	65
1	Ramesh	77	45
2	Yogesh	67	45
3	Pritesh	45	34
4	Jitesh	56	67
5	Pragnesh	67	78
6	Mantresh	45	56
7	Khanesh	87	89
8	Antresh	44	77
9	Mitesh	67	89

```
m=df[['Sub1','Sub2']].mean(axis=0)
```

```
print(m)
```

Output:

```
Sub1    62.2
Sub2    64.5
dtype: float64
```

```
m=df[['Sub1','Sub2']].mean(axis=1)
```

```
print(m)
```

Output:

```
0    66.0
1    61.0
2    56.0
3    39.5
4    61.5
5    72.5
6    50.5
7    88.0
8    60.5
9    78.0
dtype: float64
```

4.5.9.2 Median

- Median() is used to calculate the statistical median of given dataset.

- Syntax:

```
Variable_name = DataFrame.median(axis=[0/1])
```

- Example:

```
m=df[['Sub1','Sub2']].median()
```

```
print(m)
```

Output:

```
Sub1    67.0
Sub2    66.0
dtype: float64
```

```
m=df[['Sub1','Sub2']].median(axis=1)
```

```
print(m)
```

Output:

```

0    66.0
1    61.0
2    56.0
3    39.5
4    61.5
5    72.5
6    50.5
7    88.0
8    60.5
9    78.0
dtype: float64

```

4.5.9.3 Mode

- Mode() is used to calculate the statistical mode of given dataset. (most frequently occurring value)
- Syntax:
Variable_name = DataFrame.mode(axis=[0/1])
- Example:

```

m=df['Sub1'].mode()
print(m)
m=df['Sub2'].mode()
print(m)

```

Output:

```

0    67
dtype: int64
0    45
1    89
dtype: int64

```

4.5.9.4 Variance

- Var() is used to calculate the statistical variance of given dataset.
- Syntax:
Variable_name = DataFrame.var(axis=[0/1])
- Example:

```

m=df[['Sub1','Sub2']].var()
print(m)

```

Output:

```

Sub1    209.733333
Sub2    367.611111
dtype: float64

```

4.5.9.5 Standard Deviation

- std() is used to calculate the statistical standard deviation of given dataset.
- Syntax:
Variable_name = DataFrame.std(axis=[0/1])
- Example:

```

m=df[['Sub1','Sub2']].std()
print(m)

```

Output:

```

Sub1    14.482173
Sub2    19.173187
dtype: float64

```